



RECAP

From next week...

DEFINING CONSTRUCTOR VS MEMBER FUNCTION

Constructor Syntax:

```
className :: className(... )  
{  
    ...  
}
```

Member Function Syntax:

```
ReturnType className::MemberFunctionName(... )  
{  
    .....  
}
```

CONSTRUCTOR INITIALIZATION

//A constructor initializes variables to zero

```
Rectangle::Rectangle()  
{  
    width=0;  
    length=0;  
}
```

//A parameterized constructor

```
Rectangle::Rectangle(int w, int l)  
{  
    width=w;  
    length=l;  
}
```

//A constructor initializes variables to zero

```
Rectangle::Rectangle(): width(0),length(0)  
{ }
```

//A parameterized constructor

```
Rectangle::Rectangle(int w, int l): width(w),length(l)  
{ }
```

CONSTRUCTOR INITIALIZATION LIST

- A list in the definition of a constructor that specifies initial values for the data members of that class.
- Initialise its data members by invoking constructors

```
ClassName(parameterList) : member1(valueList1),member2(valueList2)...\n{\n    // ...\n}
```

CONSTRUCTOR OVERLOADING

- A constructor must have the same name as the class.
- A class can contain multiple constructors as long as they differ in their parameter lists (number, type, or order of parameters).
- This is called constructor overloading.

//Constructor signature

- ① **Rectangle();**
- ② **Rectangle(int,int);**
- ③ **Rectangle(int);**



DESTRUCTOR

A special member function that is automatically called whenever an object of its class ceases to exist.

DESTRUCTORS

- Destroys or deletes an object.
- Release any resources that the corresponding constructor allocated.
- A destructor's name is the same as the name of the class it belongs to, except that the name is preceded by a tilde symbol (~).

Syntax:

```
~constructor-name();
```

DESTRUCTOR: EXAMPLE

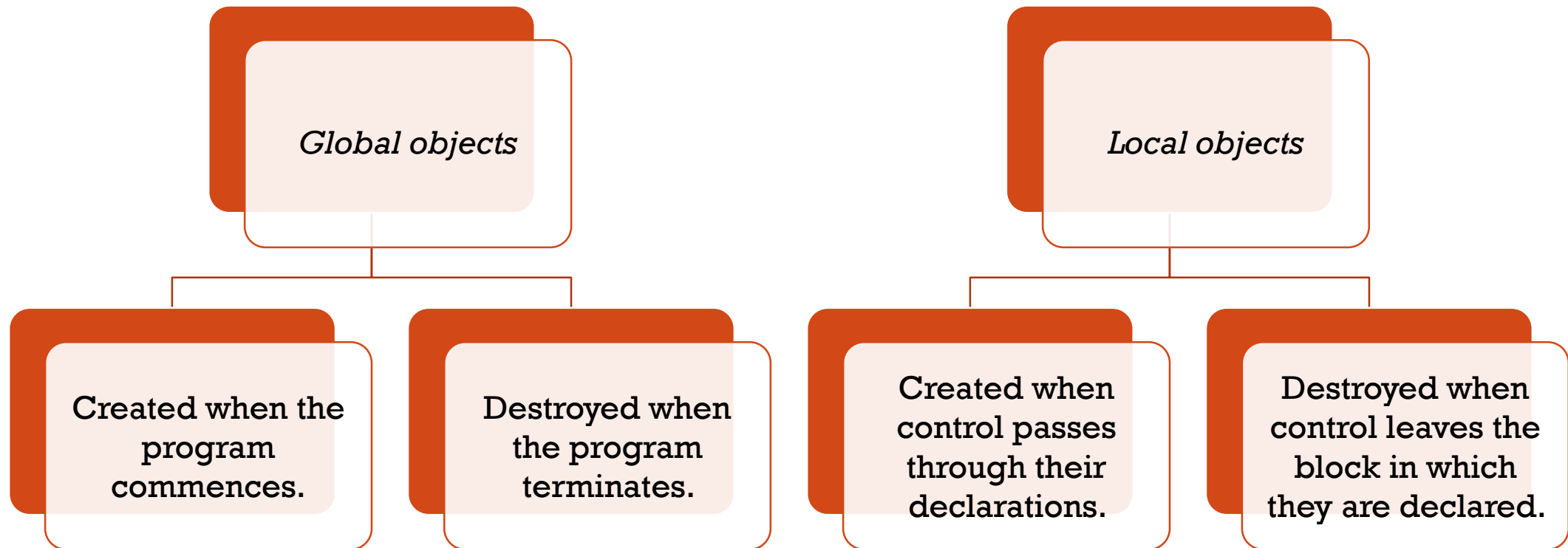
```
Constructor is called  
Hello World!  
Destructor is called
```

```
1  #include <iostream>
2  using namespace std;
3
4  class HelloWorld{
5  public:
6      HelloWorld(){
7          cout<<"Constructor is called"<<endl;}
8
9      ~HelloWorld(){
10         cout<<"Destructor is called"<<endl;}
11
12         void display(){
13             cout<<"Hello World!"<<endl;
14         }
15     };
16
17 int main(){
18     //Object created
19     HelloWorld obj;
20     //Member function called
21     obj.display();
22 }
```

DESTRUCTORS

- A destructor begins with a tilde symbol (~).
- It cannot be declared as static or const.
- A destructor does not take any parameters.
- It has no return type, not even void.
- A destructor is typically declared in the public section, but it can also be declared as protected or private depending on the design requirements.
- A class can have only one destructor.
- If no destructor is defined, the compiler automatically generates a default destructor.

OBJECT LIFETIMES VS CONSTRUCTOR/DESTRUCTOR CALL



CONSTRUCTORS / DESTRUCTORS: CALL ORDER

- Depends on order of execution (when execution enters and exits scope of objects)
- Destructor calls reverse order of constructor calls
- Global scope objects
 - Constructors are called before any other function (including main)
 - Destructors are called when main terminates (or exit function called)
 - Not called if the program terminates with abort
- Local scope objects
 - Constructors are called when objects are defined and each time execution enters the scope
 - Destructors are called when objects leave scope / execution exits block in which object defined
 - Not called if the program ends with abort



ACTIVITY

OUTPUT?

```

1 #include<iostream>
2 using namespace std;
3
4 class Rectangle
5 {
6     int width;
7     int length;
8
9     public:
10
11     1 Rectangle()
12     {
13         width=0;
14         length=0;
15     }
16
17     2 Rectangle(int w,int l){
18         width=w;
19         length=l;
20     }
21
22     3 Rectangle(int w){
23         width=w;
24         length=0;
25     }
26
27     ~Rectangle(){
28         cout<<"Destructor is called"<<endl;
29         cout<<"Object with "<<width<<" "<<length<<endl;
30     }
31
32     void getWL();
33 };
34

```

```

35 void Rectangle::getWL()
36 {
37     cout<<"Width is "<<width<<endl;
38     cout<<"Length is "<<length<<endl;
39 }
40
41 int main()
42 {
43     Rectangle Rec3;
44     Rec3.getWL();
45
46     Rectangle Rec1(4);
47     Rec1.getWL();
48
49     Rectangle Rec2(1,1);
50     Rec2.getWL();
51
52 }

```

```

Width is 0
Length is 0
Width is 4
Length is 0
Width is 1
Length is 1
Destructor is called
Object with 1 1
Destructor is called
Object with 4 0
Destructor is called
Object with 0 0

```

```
class Rectangle
```

```
{
```

```
    int width;
```

```
    int length;
```

```
    public:
```

```
        //constructor signature
```

```
        Rectangle();
```

```
        Rectangle(int,int);
```

```
        Rectangle(int);
```

```
        //destructor signature
```

```
        ~Rectangle();
```

```
        //function signature
```

```
        void getWL();
```

```
        int areaRect();
```

```
};
```

```
1  #include<iostream>  
2  using namespace std;  
3  
4  class Rectangle  
5  {  
6      int width;  
7      int length;  
8  
9      public:  
10         //constructor signature  
11         Rectangle();  
12         Rectangle(int,int);  
13         Rectangle(int);  
14         //destructor signature  
15         ~Rectangle();  
16         //function signature  
17         void getWL();  
18         int areaRect();  
19 };
```

```

21 //A constructor initializing variables to zero
22 Rectangle::Rectangle():width(0),length(0){}
23
24 //A parameterized constructor
25 Rectangle::Rectangle(int w,int l):width(w),length(l){}
26
27 //A parameterized constructor
28 Rectangle::Rectangle(int w):width(w),length(10){}
29
30 //Destructor
31 Rectangle::~~Rectangle(){
32     cout<<"Destructor is called: "<<areaRect()<<endl;
33 }
34
35 //A function to display the width and the height of the rectangle
36 void Rectangle::getWL()
37 {
38     cout<<"Width is "<<width<<endl;
39     cout<<"Length is "<<length<<endl;
40 }

```

```

42 //A function to calculate the area of the rectangle
43 int Rectangle::areaRect()
44 {
45     return width*length;
46 }
47
48 int main()
49 {
50     Rectangle Rec1(4);
51     Rec1.getWL();
52
53     Rectangle Rec2(1,1);
54     Rec2.getWL();
55
56     Rectangle Rec3;
57     Rec3.getWL();
58
59     cout<<"Rectangle 1 Area: "<<Rec1.areaRect()<<endl;
60     cout<<"Rectangle 2 Area: "<<Rec2.areaRect()<<endl;
61     cout<<"Rectangle 3 Area: "<<Rec3.areaRect()<<endl;
62 }

```



ACTIVITY :



```

1  #include<iostream>
2  using namespace std;
3
4  class CreateDestroy{
5      private:
6          int objectID;
7          string message;
8
9      public:
10         CreateDestroy():objectID(0),message(""){}
11         CreateDestroy(int, string);
12         ~CreateDestroy();
13     }obj;
14
15     CreateDestroy obj123(123,"Global Object");
16

```

```

17 CreateDestroy::CreateDestroy(int ID,string msg){
18     objectID = ID;
19     message=msg;
20     cout << "Object " << objectID << " constructor runs "
21         << message << endl;
22 }
23
24 CreateDestroy::~~CreateDestroy()
25 {
26     cout << "Object " << objectID << " destructor runs"
27         << message << endl;
28 }
29

```

```

30 int main()
31 {
32     cout << "MAIN FUNCTION: EXECUTION BEGINS" << endl;
33     CreateDestroy obj1( 1, "(local automatic in main)" );
34     CreateDestroy obj2( 2, "(local automatic in main)" );
35     CreateDestroy obj3( 3, "(local automatic in main)" );
36     cout << "\nMAIN FUNCTION: EXECUTION ENDS" << endl;
37 }

```

```
Object 123 constructor runs Global Object  
MAIN FUNCTION: EXECUTION BEGINS  
Object 1 constructor runs (local automatic in main)  
Object 2 constructor runs (local automatic in main)  
Object 3 constructor runs (local automatic in main)  
  
MAIN FUNCTION: EXECUTION ENDS  
Object 3 destructor runs(local automatic in main)  
Object 2 destructor runs(local automatic in main)  
Object 1 destructor runs(local automatic in main)  
Object 123 destructor runsGlobal Object  
Object 0 destructor runs  
  
-----  
Process exited after 1.017 seconds with return value 0  
Press any key to continue . . . |
```



ACTIVITY :

Write down the output...

```
#include<iostream>
#include<cassert>
using namespace std;

class List{
    private:
        int length;
        int *array;
    public:
        List();
        List(int size);
        ~List();

        void displayList();
        void SetList(int a);
};
```

```
int main()
{
    cout<<"START"<<endl;

    List l1;
    l1.displayList();
    l1.SetList(2);
    l1.displayList();

    List l2(3);
    l2.displayList();
}
```

```

List::List()
{
    length = 10;
    array = new int [length];
    assert(array != 0);
    for (int i = 0; i < length; i++)
        array[i] = 0;
}

List::List(int size)
{
    length = size;
    if (length <= 0)
        array = 0;
    else
    {
        array = new int [length];
        assert(array != 0);
        for (int i = 0; i < length; i++)
            array[i] = 3;
    }
}

```

```

void List::displayList(){
    for (int i = 0; i < length; i++)
        cout<<array[i]<<" ";
    cout<<endl;
}

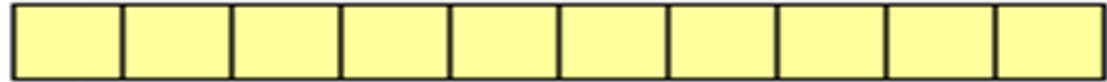
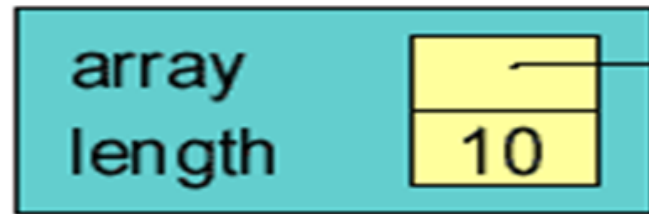
void List::SetList(int a){
    for (int i = 0; i < length; i++)
        array[i] = i+a;
}

// Destructor
List::~~List()
{
    delete [] array;
    cout<<"Called the destruct"<<endl;
}

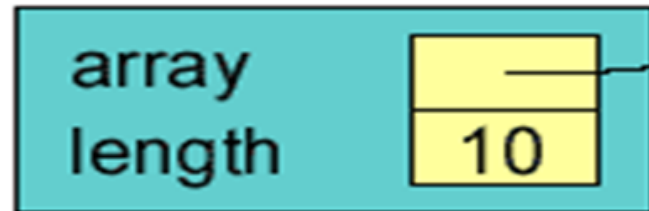
```

COPY OBJECTS

aList



Bitwise copy of aList



38

DEFAULT COPY CONSTRUCTOR

Makes a bitwise copy of the class instance

COPY CONSTRUCTOR

- An overloaded constructor that initializes an object using another object of the same class.
- Initialize the members of a newly created object by copying the members of an already existing object.
- A copy Constructor may be called, when
 - an object of the class is returned by value.
 - an object of the class is passed (to a function) by value as an argument.
 - an object is constructed based on another object of the same class.
 - the compiler generates a temporary object.
- A copy constructor cannot return a value and has no return type (not even **void**).
- Allows instantiating an object as an exact copy of another in terms of its attribute values.
- Default or User-Defined copy constructor

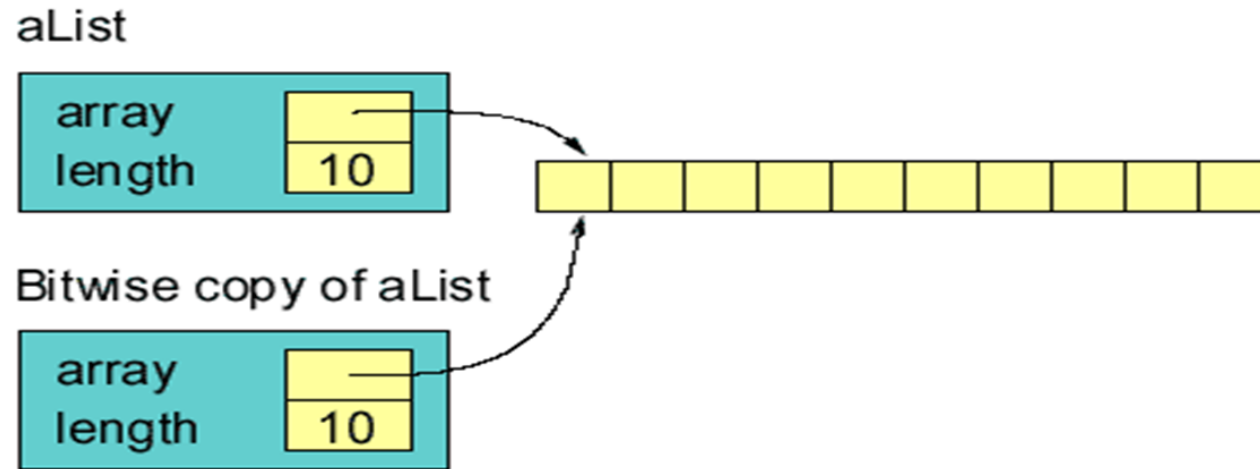
DEFAULT COPY CONSTRUCTOR

If a copy constructor for a class is NOT defined explicitly, then the compiler will *automatically* provide a copy constructor that makes a *bitwise copy* of the class instance.

Bitwise copying is a way to get a copy of a class object by copying every bit (byte) of a particular class object (instance). Bitwise copying is used when it is necessary to obtain an identical destination object from the source object.

DEFAULT VS USER DEFINED COPY CONSTRUCTOR

- Works fine in general.
- HOWEVER, if an object has pointers or any runtime allocation of the resource like a file handle, a network connection, etc.
- Need to define a user-defined copy constructor.



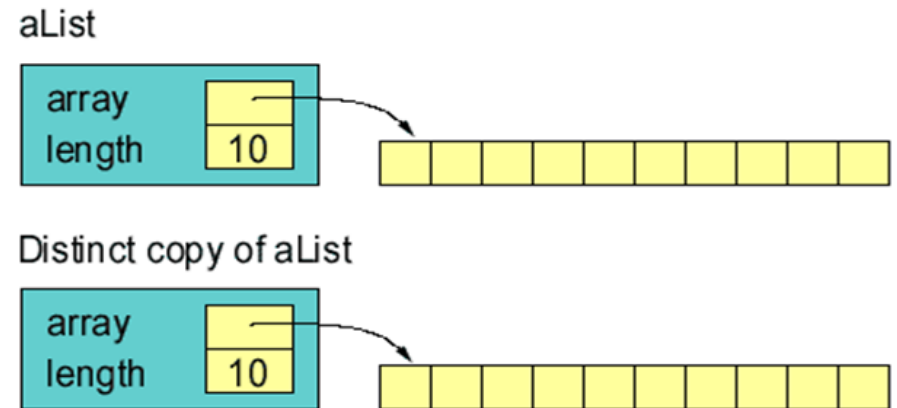
USER DEFINED COPY CONSTRUCTOR

Copy Constructor Syntax

```
className::className(const className& old_Object)
{
    // ...
}
```

USER DEFINED COPY CONSTRUCTOR

```
List::List(const List &original)
{
    length = original.length;
    if (length > 0)
    {
        array = new int [length];
        assert(array != 0);
        for(int i = 0; i < length; i++)
            array[i] = original.array[i];
    }
    else
        array = 0;
}
```



45

**WHY ARGUMENT TO A COPY
CONSTRUCTOR MUST BE
PASSED AS A REFERENCE?**

46

ACTIVITY 01: COPY CONSTRUCTOR

```
#include <iostream>
using namespace std;

class Sample {
    int id;

public:
    void init(int x) { id = x; }
    void display() { cout << endl << "ID=" << id; }
};

int main()
{
    Sample obj1;
    obj1.init(10);
    obj1.display();

    Sample obj2(obj1);
    obj2.display();
    return 0;
}
```